

ADAPTIVE MERKLE TRIE VERSIONING FOR EFFICIENT STRUCTURED DATA MANAGEMENT

BY ATHARVA SHEERSH PANDEY^{1,a}, ADHYAN JAIN^{1,b},
AND POORNIMA NEDUNCHEZHIAN^{1,c}

¹SCHOOL OF COMPUTER SCIENCE AND ENGINEERING, VIT VELLORE

^aatharva.sheersh2024@vitstudent.ac.in; ^badhyan.jain2024@vitstudent.ac.in

^cCORRESPONDING AUTHOR: poornima.n@vit.ac.in

Revon is a Python prototype for durable, linear-history key/value versioning. It stores addressed changesets alongside a fixed-depth Merkle hash trie and uses an operation-count policy to choose log aggregation or hash-pruned comparison for a version pair. After repairing JSON ownership and identity handling, we evaluated the implementation on a separate Ubuntu cloud machine. The 516 isolated executions covered synthetic histories, public taxi records and threshold controls. Seven paired trials for each of eight synthetic scenarios and one 100,000-record public scenario gave eight diff-latency intervals in favour of the hybrid over forced Merkle, with parity included in the dense case. The frozen 16,384-operation policy chose Merkle at 32,768 operations even though forced log was faster in every matched trial. These results establish neither an optimal threshold nor superiority over always-log. We also checked process telemetry and fixed-trie geometry in a separate 72-execution supplement. At a million records, one run per system verified correctness and showed that the workload could be completed; these runs do not support a comparative performance claim. Our timings include the complete Revon API and Dolt CLI workflows, so they cannot isolate storage-engine performance. We present a reproducible comparison of two correct ways to compute a diff over the same durable representation, including the measured costs and the limits of the evaluation.

1. Introduction. A store that versions structured data needs to recover exact earlier states and compare them efficiently, even when updates affect only a few records. Simply archiving files does not provide all of these capabilities. Git links immutable, content-named objects through parent commits, offering a useful starting point [10]. Those objects represent files and directories. Record stores face a related problem: keys may remain in use while update density changes, and a comparison may cover just a few edits or many commits.

We investigate whether a fixed-depth Merkle hash trie can support incremental structural sharing alongside an operation index. Revon-H chooses its diff path according to the requested interval: eligible intervals within an operation threshold use log aggregation, and the remaining requests use Merkle comparison. The trie itself has fixed geometry. Atomic, durable storage comes from SQLite. Accordingly, the efficiency referred to in the title concerns the operations and workloads measured here, without implying lower costs for structured-data management in general.

Keywords and phrases: data versioning, Merkle hash trie, hybrid differencing, content-addressed storage, benchmark methodology.

Dolt serves as our production comparator because it combines SQL with a Git-style commit graph and content-addressed Prolly Trees [11, 26]. Revon has the narrower scope of a research prototype and does not replace Dolt feature for feature. We compare calls to Revon's in-process Python API with a Dolt CLI workflow. These measurements cannot isolate the performance of the two tree structures.

1.1. Research Questions. RQ1: How does Revon-H compare with Snapshot, Log-only and Dolt in latency and storage as dataset size, mutation density and update locality vary?

RQ2: Can selection between log and Merkle diff reduce latency relative to forced-Merkle Revon-M without compromising correctness?

RQ3: What effects do branching factor and trie depth have on latency, storage and the work examined?

1.2. Contributions. The implementation uses canonical SHA-256 addresses for a fixed-depth trie. A batch affecting several keys rewrites the affected paths incrementally.

For diff, we combine addressed changesets with a tree comparison that skips subtrees whose hashes match, selecting between the two paths through a hybrid policy.

SQLite provides atomic storage for repository objects, commits, aliases, measurements and HEAD. On reopening a repository, Revon checks the hashes of the stored objects.

We evaluate five systems through six import/diff workflows on deterministic histories. Each trial includes checks of semantic outputs and earlier states. Execution order is balanced, and the repaired source is validated on a separate Linux cloud machine.

We examine how branching factor and depth affect the fixed trie. The $b=8$, $d=4$ configuration is one of the configurations evaluated; we do not presume that it is best for every workload.

2. Related Work.

2.1. Content Addressing and Persistent Structures. A root digest in a Merkle tree authenticates the larger collection beneath it [20]. Content addressing appears in Git's software histories and in IPFS's networked content [10, 5]. Persistent data structures retain earlier states through path copying [12]. For dictionaries stored on disk, later designs include ordered stratified B-trees [29] and fully persistent B-trees [8]. Revon instead routes hashed keys through a fixed-depth trie. This allows hashes to prune a comparison, but sacrifices ordered range access. Scientific array systems take a different approach: SciDB and TimeArr use layouts and deltas suited to versioned arrays [25, 28]. Version access is also supported within a main-memory database by TardisDB, and replicated versioned structures for wide-area systems are studied by Saquib et al. [23, 24]. These designs help situate Revon's choices. They do not compare its log and Merkle paths on a common history.

Merkle Search Trees use search-tree order and Merkle authentication to reconcile states [4]. Hashes also prune comparisons in Revon, but its routing is fixed-depth and its linear history is centralized, unlike a replicated CRDT. In LBFS, content-defined chunking identifies redundant content [21]. Dolt and Noms use related methods to choose boundaries. Such boundaries can improve deduplication when data shifts; ordered indexes, in turn, permit range operations. We evaluate neither of these capabilities in the Revon benchmark.

2.2. Structured Dataset Versioning. Dataset versioning must account for collaboration alongside the cost of storing and retrieving states. DataHub examines collaborative histories at scale [6]. Within a relational engine, Decibel supports branching and compares version-first, tuple-first and hybrid layouts [18]. OrpheusDB uses partitioning to improve

version retrieval when versioning is added to an existing DBMS [17]. The storage/reconstruction trade-off is formalized by Bhattacharjee et al. [7]. Delta Lake builds ACID tables using transaction logs and checkpoints [2], while more recent work selects materialized versions and deltas on a version graph subject to storage and retrieval constraints [16]. Consistent identification and description of releases is the concern of the FAIR-compliant proposal [14]. It complements work on storage but does not specify a diff algorithm. Revon's key/value prototype covers a narrower problem and does not address these broader requirements of dataset management.

2.3. System Design Gap. Noms stores structured values in a Merkle DAG, with support for diff and synchronization [3]. In ForkBase, content addressing is combined with fork and deduplication semantics [30]. Dolt offers ordered indexing resembling a B-tree, structural sharing and native diff through content-addressed Prolly Trees [11, 26]. Git4Data supports snapshot, branch, diff and merge through SQL over a relational engine [15]. The manuscript available for that system is an arXiv preprint, so its reported results are not treated here as peer-reviewed or independently reproduced evidence. Revon's design is more limited. It uses fixed-depth hash routing in a single-writer key/value store with linear history, without SQL database capabilities or a branching system.

The systems and studies closest to the implemented design are summarized in Table 1.

TABLE 1
Related-work summary

System or study	Representation or focus	Relevance to Revon
Persistent DS [12]	Path copying	Formal structural-sharing basis
Git [10]	Object DAG	Identity and commit ancestry; file-oriented
IPFS [5]	Merkle DAG	General content-addressed versioning
MST [4]	Merkle search tree	Authenticated ordered reconciliation
DataHub [6]	Version graph	Collaborative dataset motivation
Decibel [18]	Relational layouts	Native database branching
OrpheusDB [17]	Partitioned versions	Storage/retrieval optimization
Delta Lake [2]	Log/checkpoints	ACID time travel at data-lake scale
Noms [3]	Merkle DAG	Structured content addressing
ForkBase [30]	Forkable CAS	Fork and deduplication semantics
DoltHub [11, 26]	Prolly Trees	Mature production comparator

2.4. Design Implications and Research Positioning. Retaining history, creating versions and later reconstructing or comparing them each impose costs. Snapshots make retrieval straightforward by copying even the unchanged records. Logs postpone work until replay or checkpointing. Path copying retains the unchanged structure and creates only the paths that changed [12, 7]. Physical layout influences version creation and retrieval in Decibel and OrpheusDB [18, 17], while graph-based analyses express the storage/retrieval trade-off directly [16]. Reproducibility studies address the preservation of builds and artifacts for inspection [1]. Preserving them does not demonstrate that a storage algorithm runs faster, so we report import, commit, checkout, diff and storage separately.

The meaning of a change depends on the research question. Differential dataflow updates query results incrementally as timestamped inputs change [19]. Data provenance explains why and where output facts originated [9]; DSLog records fine-grained array lineage and supports queries over it [31]. Explain-Da-V explains semantic dataset changes through data transformations [27]. Baseline's operation-based differencing accommodates refactorings

and schema changes [13]. Revon's output identifies keys that were added, removed or modified between two states. It does not explain causes, transformation lineage, query effects or schema intent. Although these studies help position the work, their outputs and workloads do not permit direct timing comparisons with Revon's key/value diffs.

We compare two correct diff paths over the same durable history, choosing a path from properties of the requested interval. Every Revon-H commit retains a Merkle root and an addressed changeset, so selection leaves version semantics unchanged. Our hypothesis is that log aggregation works well for short, sparse intervals, whereas hash pruning may be useful after longer sequences of operations. Whether the paths cross in performance requires measurement; the switching rule alone cannot establish it. Git4Data illustrates continuing work on database-native version control [15]. Our contribution is confined to adaptive differencing over a fixed-depth hash trie, with Snapshot and Log-only serving as representation baselines. Revon is not a general-purpose data-versioning platform. The public-record extension uses generated updates to one release, and does not establish performance across diverse datasets, independent hardware, concurrent workloads or production systems.

3. Revon Design. The architecture and the scope implemented in this study are shown in Figure 1.

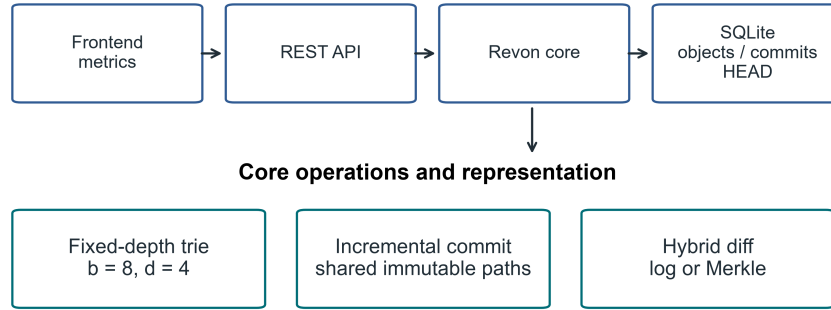


FIG 1. Revon architecture and the implemented research scope

3.1. Addressed Object Model. Trie nodes and changesets are serialized as canonical JSON, with each object's SHA-256 digest serving as its name. The commit digest includes the root, parent, changeset, message and timestamp. Readable labels such as v1 and v2 are aliases and have no effect on object identity. The current commit is identified by a durable HEAD reference.

3.2. Fixed-Depth Merkle Hash Trie. The main experiment uses $b=8$ and $d=4$. At each level, routing takes three bits from SHA-256(key); twelve bits select among 4,096 possible leaf buckets. Because a bucket retains the full keys, a shared routing prefix does not make two keys indistinguishable. With uniform hashes, expected bucket occupancy is N/b^d . We evaluate other b,d pairs separately and make no assumption that the default is optimal.

3.3. Incremental Hashing and Commit. Revon first canonicalizes a mutation batch and routes its keys, grouping updates that share a trie prefix. It rewrites each affected leaf once

and rebuilds the ancestors on those paths. Unchanged child hashes are carried into the new internal nodes, allowing the unchanged structure to be shared. One SQLite transaction writes the resulting objects and changeset, commit metadata, version alias and measurements, and updates HEAD. When the repository is reopened, Revon verifies stored payload hashes and references.

3.4. Hybrid Diff. The log path walks the ancestry of the requested interval and combines operations for each key. The Merkle path compares the roots recursively: equal hashes end that part of the search, while differing paths lead to examination of their entries. Before choosing a path, Revon-H validates both version identifiers, checks ancestry in each direction and counts the changeset operations. An available ancestor path with a count no greater than T is eligible for log aggregation. Otherwise, Revon-H uses Merkle comparison. For a reverse request on the log path, it exchanges old and new values. A version compared with itself gives an empty diff. Forced log rejects unrelated versions, and an unknown version raises an error. Revon-M always requests the Merkle path, although its public API for a version pair still checks ancestry. Both paths return the same sorted semantic diff; commit and checkout are separate operations.

We retain $T=4,096$ in the main Linux workflow matrix, as specified in the original workflow protocol. A separate Linux control tests the earlier frozen candidate, $T=16,384$. The threshold counts operations, whereas b^d specifies bucket capacity. We tune neither parameter from the Linux results.

3.5. JSON value ownership and identity. Revon accepts JSON nulls, booleans, finite numbers, strings, lists and objects whose keys are strings. It validates and detaches input containers before retaining them, and returns detached containers through read and diagnostic views. Immutable scalars may be shared. Before any write, it rejects Python-only values that are unsupported, non-string keys in nested objects, non-finite numbers and cyclic containers. Value equality follows the existing v1 JSON encoding, which uses sorted object keys, compact separators and UTF-8, with NaN excluded. The encodings of true, 1, 1.0 and signed floating zero therefore stay distinct. These rules define the Python encoding contract; they do not assert conformance to a cross-language canonical-JSON standard.

Algorithm 1 (batch update). Validate the inputs and take ownership of their values. Before interning, reject overlapping puts/deletes and deletions of missing keys. Read the old values and keep only changes to encoded identity. Group these effective mutations by routing prefix, then rewrite each touched leaf, sort its entries and hash the complete payload. Rebuild the affected ancestors while reusing unchanged child hashes, and record the addressed changeset and commit. The ownership invariant prevents callers from changing retained values through earlier inputs or supported reads.

Algorithm 2 (log composition). Traverse the ancestor interval chronologically. For each key, keep the existence/value at its first old endpoint and at its last new endpoint. Drop that key only if both endpoints agree in existence and encoded value. Thus, inserting and then deleting a key cancels, as does updating and then restoring its value; a null value is still distinct from an absent key. Sort the remaining keys. A reverse request exchanges the endpoints and swaps the added/deleted labels.

Algorithm 3 (selection). Validate the versions and check ancestry in either direction. Add the changeset operation counts across the interval. Choose log for an eligible interval whose count is at most T , and Merkle otherwise. Merkle comparison skips addressed subtrees with equal hashes and compares values in unequal leaves. Both paths use the same encoded-identity rule and return detached outputs.

Cost model. Let N denote the record count, d the depth, b the branching factor, m the keys in a batch, h the interval length, p the accumulated operations, u the distinct interval keys and s the serialized output bytes. Expected leaf occupancy under uniform routing is $N/(b^d)$, with N as the worst case. Processing a batch involves validating and copying inputs, performing m routed lookups of old values and scanning the occupied leaves. Touched leaves must be sorted and serialized, and affected internal child maps must be copied. The work therefore cannot be bounded by a dataset-independent $O(d)$.

Log composition includes ancestry traversal and $O(p + u \log u)$ work, together with encoded-value comparisons and output copying/serialization. For Merkle comparison, the work depends on the node pairs visited, sorting child slots and the leaf entries examined, as well as comparing values and sorting the output. Both public version-pair modes traverse ancestry. A reverse request may initially walk to genesis in the direction that fails. Producing the output costs at least an amount proportional to s . Larger JSON values introduce comparison and copying costs beyond those characterized by scalar-only benchmarks.

Historical nodes, changesets and commits are loaded eagerly from SQLite into memory. Although the repository stores its objects durably, they are resident while Revon runs, and the retained history must fit in memory. The amount of retained payload determines the work of opening and verifying a repository. No disk-page cache or eviction policy is provided. The atomicity of SQLite persistence does not demonstrate the production concurrency or operational guarantees that we have not tested.

Figure 2 traces the decision made for a version pair, showing the eligibility checks and the treatment of reverse requests.

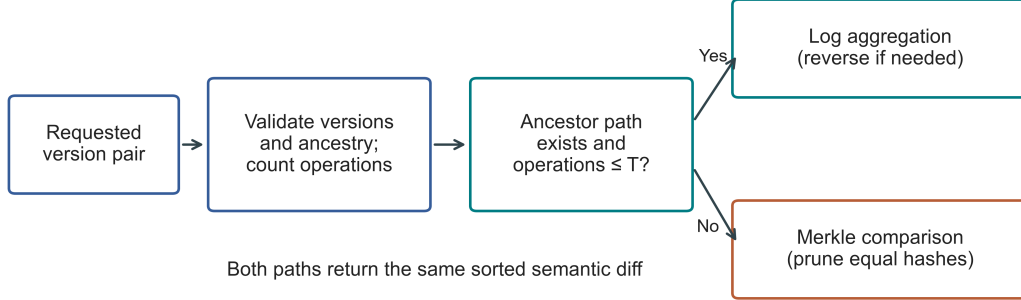


FIG 2. Actual hybrid selection for a version pair; T denotes the operation threshold. Invalid versions raise an error before selection. Both paths share the persistent trie

4. Experimental Methodology.

4.1. Environment and evidence boundaries. We ran the current evaluation on a separate Google Compute Engine n2-standard-4 VM in us-east1-c. Table 2 describes this environment and distinguishes the two source snapshots. Earlier Windows and WSL results are kept in the archive, without pooling them with Linux trials or presenting them as measurements of the repaired source. This adds a hardware environment to the evaluation. It does not provide an independent research team or isolate the effect of Linux.

TABLE 2
Linux environment and separately retained evidence

Property	Recorded configuration
Machine	GCP n2-standard-4; Intel Cascade Lake; 4 vCPUs; 16 GB RAM; us-east1-c
Operating system	Ubuntu 24.04.5 LTS; x86_64; kernel 7.0.0-1011-gcp
Storage	100 GB balanced persistent disk; no swap configured
Runtime	CPython 3.14.3, Clang 22.1.1 build; GIL enabled; JIT not enabled; SQLite 3.50.4
Comparator and packages	Dolt 2.3.1; psutil 7.1.0; PyArrow 25.0.1
Primary source	linux-validation-20261008: archived repaired source, fixed before measurement
Supplement source	linux-final-supplement-20261008: same core; zero-aware I/O sampler and geometry history checks
Background conditions	Google Ops Agent active; filesystem cache not cleared; monitoring included

To identify what was run, each bundle contains the executed source and its hashes, environment records, frozen workload definitions, execution order, raw rows and audits. We set `PYTHONHASHSEED=20261008` and archived the working revision itself, since repository HEAD alone would not identify it. Before timing on the VM, the primary campaign passed 67 unit tests and the supplement passed 69. The first supplement preflight had failed: the transferred files did not include the `test_data` directory. We created that empty fixture directory and reran the full test suite before measuring the supplement. The retained records include this packaging failure.

4.2. Variants and workloads. The variants retain history in different ways. Snapshot stores a complete state at every version; Log-only stores the starting state and subsequent operations. Revon-M requests Merkle diff, whereas Revon-H chooses its diff path. Dolt uses a primary-key SQL table and returns native JSON diff. For Dolt, SQL INSERT and CSV table import are timed separately, giving six workflow variants across five systems. Their later operations follow the same semantic contract, although their interfaces and storage features differ. The figures use the workload identifiers assigned in Table 3.

TABLE 3
Linux workloads; all primary histories have ten generated commits

ID / scenario	Rows	Changes / commit	Locality
S1 Small sparse	1,000	10	Spread
S2 Medium sparse	10,000	10	Spread
S3 Medium dense	10,000	1,000	Spread
S4 Large sparse	100,000	100	Spread
S5 Application-key local	100,000	100	Lexically adjacent keys
S6 Range local	100,000	100	Contiguous key intervals
S7 Repeated key	100,000	100	Repeated small key set
S8 Hash-route local	100,000	100	Shared 9-bit hash prefix
P1 TLC public records	100,000	100	Generated fare corrections

Eight synthetic scenarios, each with six variants, two warm-ups and seven measured trials, account for 432 executions. The public-record cases contribute 27 executions across Revon-M, Revon-H and Dolt SQL, and the two threshold controls contribute another 54.

Three million-row diagnostics bring the campaign to 516 executions: 114 warm-ups, 399 measured trials and three diagnostics. All 516 finished with verified outputs. Failed timing executions are not silently replaced or omitted.

4.3. Public dataset and generated histories. The public-data extension uses NYC TLC's January 2024 yellow-taxi release [22]. A checksum fixes the Parquet input, which has 2,964,624 rows. With seed 20261001, we use a dispersed affine permutation to obtain nested subsets of 10,000, 100,000 and 1,000,000 rows. We retain eleven fields: vendor, pickup/drop-off times, passenger count, distance, pickup/drop-off locations, payment type, fare, tip and total. Each key is formed from the release, the file-checksum prefix and the physical row ordinal. It identifies a row within this fixed file, not the same trip in different releases. Values, including nulls, are serialized as compact UTF-8 JSON with sorted keys. In the million-row subset, 47,277 passenger counts are missing, and serialization succeeds for every row. Identical field values do not cause rows to be combined. The affine permutation used for sampling is not uniformly random.

We generate the public edit histories with ten commits. On the 100,000-row subset in the Linux campaign, a commit makes 100 corrections; in the million-row diagnostic, it makes 1,000. A correction adds 0.01 to both fare and total, leaving the other retained fields untouched. All systems receive identical records and mutations. These edits are constructed for the experiment and are not observed longitudinal changes. The public-data builder ignores the inherited `payload_bytes=8` field in the configuration. All eleven fields remain in a public value, and payload size is accounted for through the measured serialized bytes. The harness stores a canonical record as a JSON-encoded string in the common value column. We therefore do not measure native mutation of nested objects or queries on individual fields.

TABLE 4
Checksum-pinned public data and Linux protocol

Item	Definition
Source	NYC TLC yellow_tripdata_2024-01.parquet; 2,964,624 source rows
Raw SHA-256	c4d59da7bbc8abaeceb1727947ee93d9891a71acb42854bd80db1571b2030510
Prepared million-row SHA-256	6608b7680e82432ca22f90a71d4994676ffd290c4fc85f17f91fab03d4c7b661
P1	100,000 records; H=10; C=100; 2 warm-ups + 7 measured trials per system
Scale diagnostic	1,000,000 records; H=10; C=1,000; one trial per system; no interval or ranking

4.4. Timing, correctness and uncertainty. For every primary execution, we start a fresh worker and create a repository. The protocol records model order, which is shuffled deterministically and rotated within repetition blocks. Import timing ends when the first durable version has been created. The reported incremental-commit time is the median of the individual commit timers for that history. A diff runs from the first version to the last and returns sorted changed keys, together with old/new value hashes, as canonical JSON. Checkout writes the complete sorted key/value state in canonical UTF-8 JSON. A Dolt timer includes launching the CLI, parsing JSON and normalizing the output. Revon is called through its in-process API. This difference in timing boundaries prevents a speed claim about the engines alone.

We perform correctness checks outside the operation timers. In every primary execution, we verify the hashes of the initial and final states, check all H+1 historical states, and

compare the observed canonical semantic diff with a workload oracle constructed independently. The verification JSON retains observed and expected hashes, rather than just an expected count of changed keys. The post-run audit regenerates workloads and checks the exact matrix, the selected paths and the identity of the source, data and protocol. It also recomputes all summaries. We audit the downloaded evidence locally as well. These procedures check internal integrity; they do not constitute an independent reproduction of the timings.

We exclude warm-ups and report the median and interquartile range (IQR) of the seven measured repository trials. For a latency ratio, we first pair the trials, then report the median ratio and a paired percentile bootstrap interval from 20,000 resamples. Values above one favour the denominator. These exploratory 95% intervals describe repeated timings of a fixed workload on one VM. They are not population confidence intervals, and we apply no multiple-comparison correction. Taking the median of paired ratios need not give the same value as dividing the two latency medians. We retain every completed trial, including timing outliers.

The parent process samples the worker and its descendants every 10 ms, recording RSS, CPU and I/O counters. Peak measurements include workload generation, the oracle state and verification along with the storage work. They cannot be assigned to the storage engine alone. A short peak or a short-lived subprocess's counters can fall between samples. The original sampler treats an all-zero read counter as missing, so we draw no read-I/O conclusion from the primary campaign. In a separate supplement, we distinguish a recorded zero from an unavailable counter and retain those data apart from the primary results. A cached workload can examine logical data while recording zero physical read bytes.

We measure repository footprint as directory size before closing or compacting it. This includes indexes, metadata and the costs of the representation. Logical payload normalization and selected compaction probes from the first trial are also retained. These measurements describe the workflow rather than isolating tree overhead. CPU, RSS and I/O span the full worker lifecycle, giving them a different scope from the individual operation timers.

4.5. Frozen threshold and geometry controls. The earlier Windows study evaluated thresholds of 1,024, 2,048, 4,096, 8,192 and 16,384 operations on predefined calibration and validation profiles that did not overlap. Minimizing normalized calibration latency selected $T=16,384$ before validation. Its held-out normalized score was 1.032, against 1.297 for $T=4,096$; always-log scored 1.000 in both splits. These cached-query measurements belong to their archived source and environment, and do not measure the repaired Linux implementation. Selection of the largest candidate in the search does not identify an optimum. The earlier long-history and three-seed pilots are retained as exploratory supplementary evidence.

In the Linux threshold control, $T=16,384$ remains frozen. We use 10,000 rows and 512 changes per commit, with $H=16$ and $H=64$ giving 8,192 and 32,768 accumulated operations. Forced log, forced Merkle and hybrid are run in separate workers, each with two warm-ups and seven measured trials. We do not change the threshold in response to Linux results. These two cases check how the existing choice carries over to Linux, without constituting a new calibration/validation cycle.

The geometry supplement uses a single spread workload: 10,000 rows, ten commits, 100 changes per commit, 32-byte synthetic values and seed 20260824. Diff is fixed to forced Merkle for b4-d6, b8-d3, b8-d4, b8-d5 and b16-d3. Each configuration runs in a fresh worker for two warm-ups and seven measured trials. We rotate the order to limit confounding between configuration and time. Every execution checks the eleven historical

states and endpoint diff outside the timers. Geometry timings follow the same adapter output contract, but the resource peaks measured by its inherited in-process monitor are kept separate from the external RSS measurements.

5. Linux Results.

5.1. Hybrid versus forced Merkle. Revon-M and Revon-H store the same persistent representation and changesets. The ablation therefore changes the requested diff strategy. Figure 3 and Table 5 report all nine comparisons with repeated trials. Eight hybrid intervals exclude parity, while the interval for dense S3 includes one. The sparse and repeated-key cases show the benefit of access to log aggregation under these conditions. They do not demonstrate that switching is universally advantageous or that it saves storage at the representation level.

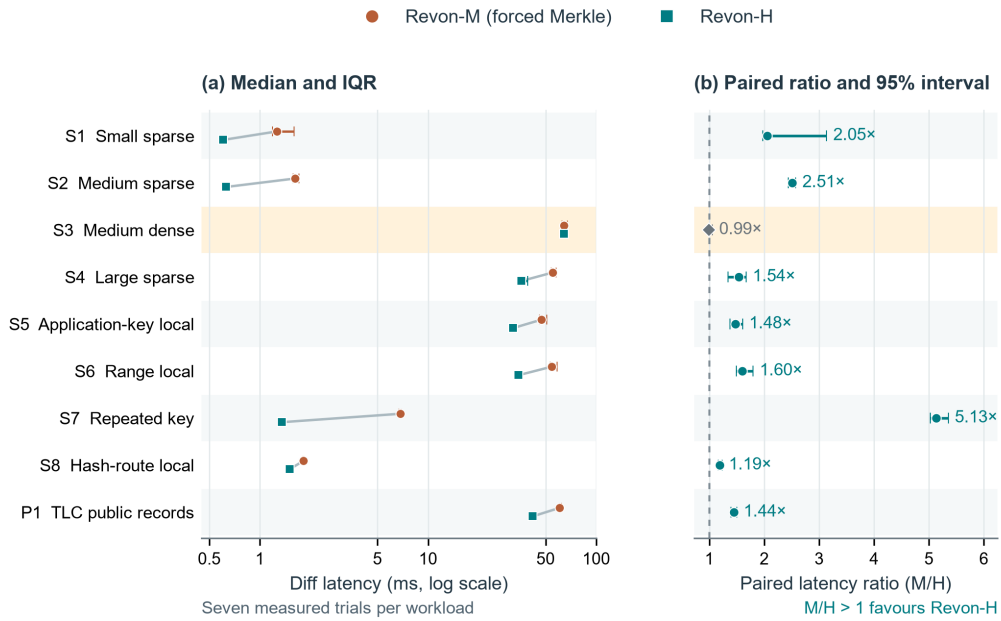


FIG 3. Linux Revon-M and Revon-H diff comparison. (a) Median latency with interquartile ranges on a logarithmic axis; paired medians are connected within each workload. (b) Median matched-trial M/H latency ratio with paired percentile bootstrap 95% intervals (20,000 resamples); the dashed line denotes parity. Each workload has seven measured trials. The highlighted S3 dense case has an interval containing parity; the other eight favour H. Ratios are medians of matched-trial ratios, not ratios of the two latency medians. Workload definitions are in Table 3. These intervals concern repeated timings for fixed histories on one VM, not independent datasets or machines.

TABLE 5
Paired M/H diff ablation; milliseconds and median trial ratio [95% interval]

ID	M (ms)	H (ms)	M/H ratio [interval]	H path
S1	1.259	0.604	2.05 [1.97, 3.14]	log
S2	1.620	0.629	2.51 [2.44, 2.57]	log
S3	64.135	64.183	0.99 [0.97, 1.06]	merkle
S4	54.897	35.749	1.54 [1.34, 1.67]	log
S5	47.253	31.967	1.48 [1.38, 1.60]	log
S6	54.352	34.537	1.60 [1.50, 1.80]	log
S7	6.862	1.345	5.13 [5.02, 5.35]	log
S8	1.817	1.503	1.19 [1.17, 1.23]	log
P1	60.597	41.964	1.44 [1.39, 1.50]	log

In P1, the median diff latency is 41.964 ms for Revon-H and 60.597 ms for forced Merkle. Both process the same eleven-field public records with the same generated updates. S3 reaches 10,000 accumulated operations, causing H to use Merkle. Its near-parity result is consistent with the variants sharing a diff path, with selection work added for H.

5.2. *Threshold control and adverse result.* Both threshold workloads appear in Figure 4 and Table 6, including the case that contradicts the frozen policy's choice. H uses log at 8,192 operations. At 32,768 operations it switches to Merkle, despite forced log being faster in each of the seven matched measured trials. Operation count by itself therefore fails to select the fastest path in these cases. The Linux control fulfils the validation task, but the optimal threshold remains unresolved.

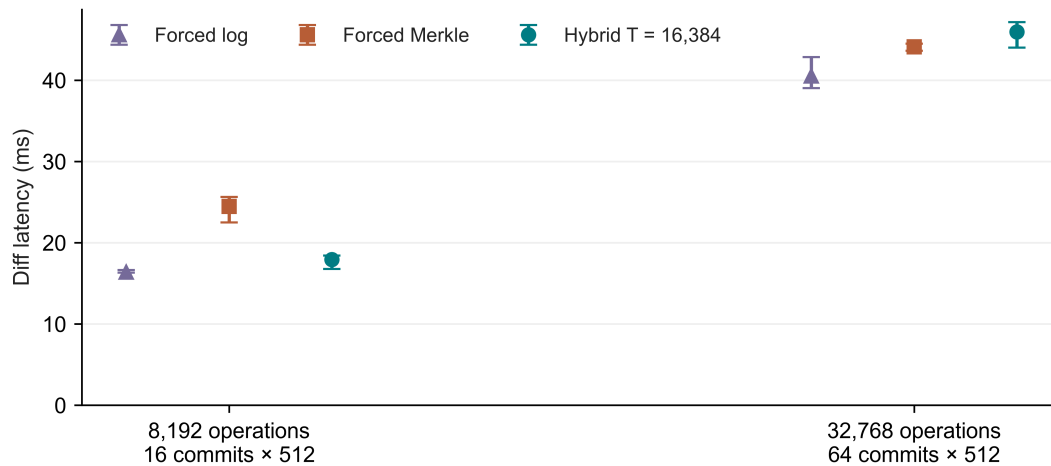


FIG 4. Linux control for the previously frozen $T=16,384$ policy; medians and IQRs from seven measured trials per path, with different history lengths labelled explicitly

TABLE 6
Frozen threshold controls; diff medians in milliseconds

Operations / history	Forced log	Forced Merkle	Hybrid	Selected path
8,192 / H=16	16.364	24.466	17.926	log
32,768 / H=64	40.520	44.120	45.944	merkle

5.3. Complete workflow comparison. Table 7 compares the canonical diff outputs of the Revon-H API and Dolt SQL/CLI workflows. Each of the nine paired Dolt/H intervals lies above one. This finding applies to the complete workflows. The measurements combine data-structure costs with process startup, language, serialization, query execution and resident state. Figure 5 presents commit and checkout separately, preventing the diff result from being interpreted as an advantage for every operation.

TABLE 7
Dolt SQL/CLI versus Revon-H diff; milliseconds and paired ratio [95% interval]

ID	Dolt (ms)	H (ms)	Dolt/H ratio [interval]
S1	90.704	0.604	148.15 [139.77, 160.15]
S2	95.515	0.629	148.70 [140.08, 150.80]
S3	297.617	64.183	4.70 [4.54, 4.77]
S4	152.472	35.749	4.25 [3.84, 4.34]
S5	125.594	31.967	3.73 [3.71, 3.87]
S6	135.765	34.537	3.88 [3.69, 4.12]
S7	97.451	1.345	71.73 [68.95, 74.32]
S8	110.966	1.503	71.24 [68.83, 77.06]
P1	168.576	41.964	4.02 [3.77, 4.07]

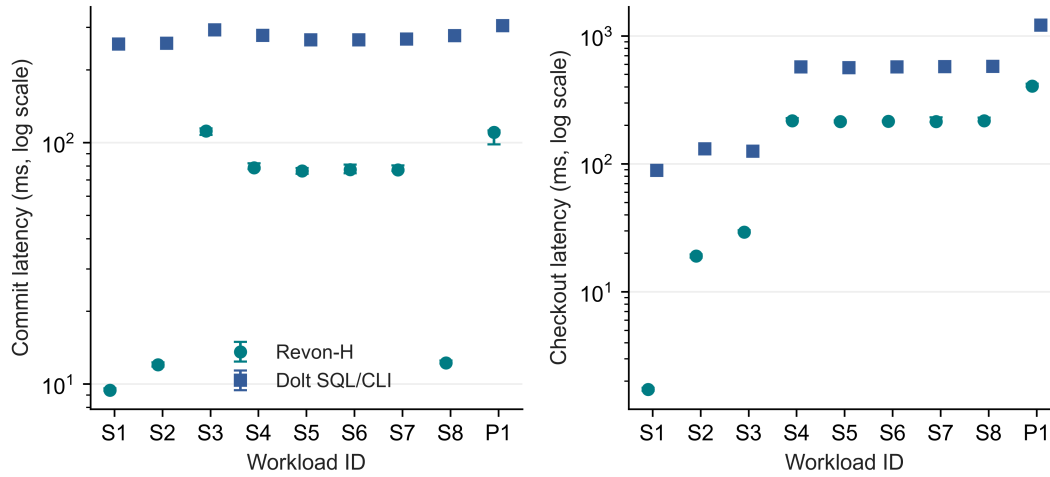


FIG 5. Linux incremental-commit and full-checkout latency for Revon-H and Dolt SQL/CLI; points show medians and bars show IQRs of seven trials, using logarithmic latency

5.4. Storage and memory costs. Figure 6 and Table 8 place the latency results alongside repository footprint and RSS for the complete worker. Addressed changesets are retained by both M and H. Their small measured directory-size difference arises from the workflows: taking the log path does not discard trie data. Dolt uses different serialization, compression and indexing as well. A fast diff can therefore come with a much larger repository. The reported resource costs are part of the assessment of efficiency.

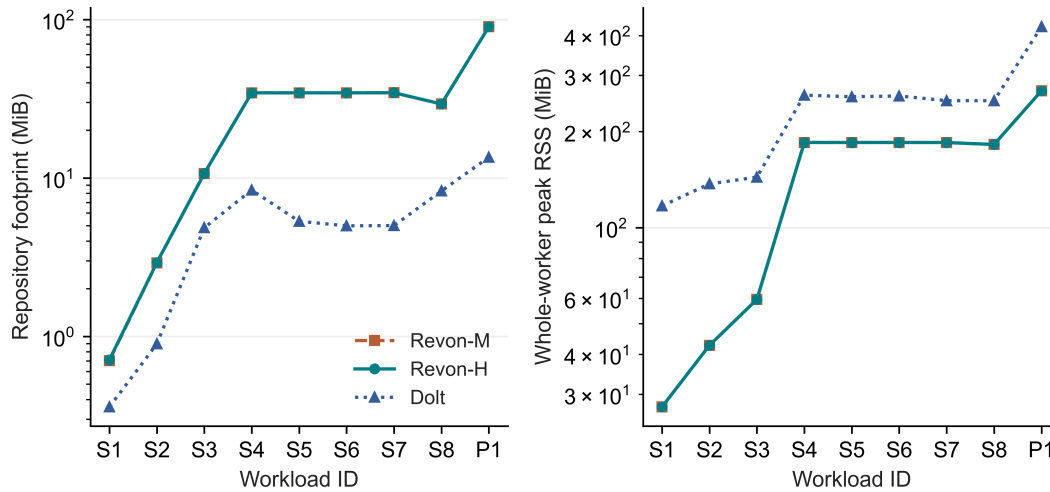


FIG 6. Median repository directory size before compaction and median whole-worker peak RSS across seven Linux trials; logarithmic scales, including shared workload and verification memory in RSS

TABLE 8
Repository footprint and worker RSS; medians in MiB

ID	M disk	H disk	Dolt disk	H RSS	Dolt RSS
S1	0.7	0.7	0.4	27.4	116.9
S2	2.9	2.9	0.9	42.7	137.1
S3	10.7	10.7	4.8	59.5	143.9
S4	34.4	34.4	8.4	184.9	260.1
S5	34.4	34.4	5.3	184.8	257.2
S6	34.4	34.4	5.0	184.9	258.6
S7	34.5	34.5	5.0	184.8	250.0
S8	29.3	29.3	8.3	182.2	249.9
P1	90.1	90.0	13.5	268.6	426.5

5.5. Million-record feasibility. At a million rows, we create and verify one fresh repository for each of M, H and Dolt. Verification checks all eleven historical states and the common diff oracle. Table 9 reports the individual observations, with no IQRs, confidence intervals or performance ranking. Because the interval contains 10,000 accumulated operations, exceeding $T=4,096$, H uses Merkle. Each run demonstrates completion of this bounded workload on the VM. One run per system cannot establish reliable relative scaling or identify an optimal configuration; repeated million-row trials are still needed.

TABLE 9
Single-run million-record diagnostics; $n=1$ per system, no comparative inference

System	Import (s)	Commit (ms)	Diff (ms)	Checkout (s)	RSS (GiB)
Dolt	24.902	547.311	817.034	11.454	2.555
Revon-H	32.289	2961.901	1233.601	4.729	2.561
Revon-M	32.454	2979.717	1265.367	4.685	2.561

5.6. Fixed-trie geometry and telemetry. Figure 7 and Table 10 report the geometry supplement. Three configurations, b4-d6, b8-d4 and b16-d3, each have 4,096 leaf buckets. Their expected occupancy is consequently equal, and comparing them examines the effect of routing shape. When $b=8$ is held fixed, changing depth changes bucket capacity as well as the balance between traversal and leaf scanning. These observations come from one workload on one VM and cannot identify a geometry that is optimal in general.

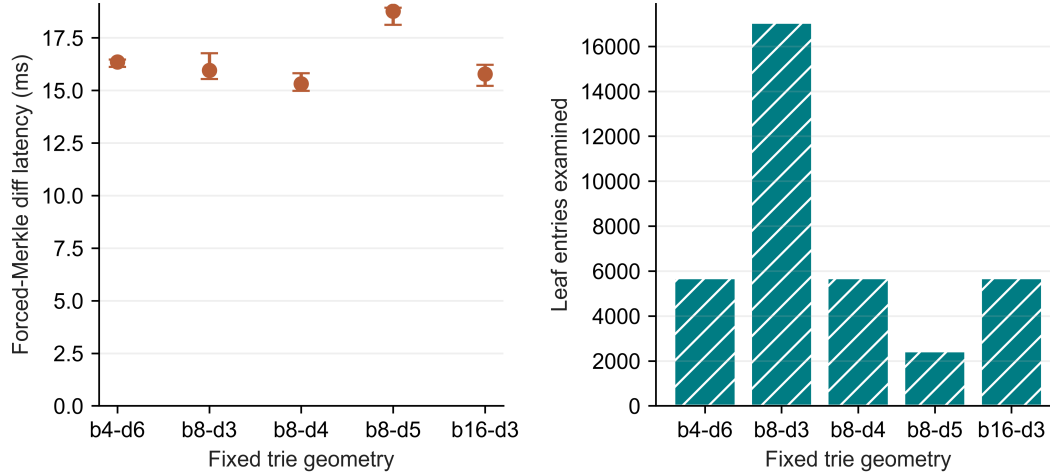


FIG 7. Linux fixed-trie geometry supplement; left shows forced-Merkle diff medians and IQRs from seven trials, right shows median examined leaf entries

TABLE 10
Geometry work and footprint; one fixed spread workload

Geometry	Buckets	Diff (ms)	Leaf entries	Node pairs	Disk (MiB)
b4-d6	4096	16.355	5682	3628	4.32
b8-d3	512	15.949	17048	585	6.88
b8-d4	4096	15.316	5682	3729	4.25
b8-d5	32768	18.765	2426	6089	6.18
b16-d3	4096	15.775	5682	3929	8.12

We checked the telemetry regression with real isolated workers, testing both valid all-zero I/O samples and unavailable counters. Both tests passed. The subsequent Linux supplement consists of 27 executions on a fixed 10,000-row workload: M, H and Dolt each have two warm-ups and seven measured trials. The recorded rows preserve zero read bytes as a measured value rather than leaving them blank. This confirms how the counters are handled. It does not establish cold-storage behaviour or eliminate sampling loss, and it leaves the primary latency summaries unchanged.

5.7. Representation baselines. For the representative S4 spread workload, Table 11 gives descriptive medians for all six workflows. The complete eight-scenario synthetic matrix is retained in the GitHub reproducibility archive (Appendix A), along with trial dispersion, normalized storage and the separate SQL/CSV import variants. In the common harness, Snapshot and Log-only control for the choice of representation, without offering the same production features. Their results show the cost of keeping complete states or replaying operations, to be considered alongside the costs measured for Revon.

TABLE 11

Representative S4 workflow medians; complete eight-scenario matrix in the GitHub reproducibility archive

Workflow	Import (s)	Commit (ms)	Diff (ms)	Checkout (ms)	Disk (MiB)
Snapshot	0.071	69.577	624.860	168.158	53.5
Log-only	0.072	2.474	16.483	159.538	5.0
Revon-M	1.488	78.842	54.897	215.661	34.4
Revon-H	1.644	78.590	35.749	217.482	34.4
Dolt SQL	1.663	277.593	152.472	572.585	8.4
Dolt CSV	1.335	273.425	149.325	559.342	7.9

6. Discussion. RQ1: the Linux results show diff-workflow advantages in the measured cases and report the associated storage, memory and commit costs. The complete raw matrix includes Snapshot, Log-only and both Dolt import paths, even when the figures focus on M/H/Dolt. All comparisons use common materialized outputs within a bounded versioning workflow with a single writer. They do not establish better SQL query processing, transaction concurrency, relational functionality or structured-data management in general.

RQ2: log aggregation improves on forced Merkle in eight of the nine paired cases with repeated trials. The dense case remains inconclusive. Always-log is still needed as a static control. The adverse result for the frozen threshold demonstrates that improvement in an M/H ablation alone cannot validate adaptation. Developing a richer selector would require fresh calibration histories followed by untouched validation histories. We do not use the present results to implement or tune such a selector.

RQ3: the geometry supplement provides Linux timings and work counters for the current source with execution order rotated. Bucket occupancy accounts for only part of the work. Serialization of touched leaves, reconstruction of ancestors, output size and traversal shape also affect cost. Equal-capacity configurations have equal expected occupancy and must be described accordingly.

The JSON repair is included in every current Linux experiment. Previously, mutable aliases could remain connected to stored values. Python equality could also treat true as equal to 1, or 1 as equal to 1.0. We repaired those semantics by detaching inputs and outputs and comparing v1 encoded identities, keeping the object encoding and on-disk schema intact. Archived earlier timings describe the implementation before this repair. The Linux trials exercise the repaired source on the supplied histories, and unit tests cover null/absence, nested ownership, unsupported values and rollback.

7. Limitations And Future Work. External validity: we used a cloud VM different from the original Windows/WSL host, but all Linux repetitions were performed by the authors on that one hardware configuration. The hardware, operating system, interpreter build, virtualization and storage changed together. We cannot attribute timing differences to Linux alone or extend them to other machines. Google Ops Agent ran during the experiments, and filesystem caching remained active. The measurements therefore do not represent an entirely idle machine or cold I/O.

Workload validity: only one deterministic history is used for each synthetic scenario. The TLC extension supplies records from a single monthly release, from which we take nested affine samples and generate corrections. It includes no naturally occurring edit histories, relational queries or multi-table joins, and does not cover diverse public datasets. Each Linux case at a million rows is a single execution, establishing feasibility only. Additional

seeds, observed histories, datasets and repeated trials at scale are needed to support broader conclusions.

Implementation scope: Revon uses a trie of fixed geometry and loads retained nodes, changesets and commits eagerly, without evicting them from memory. It supports a single writer and linear history. We have not evaluated named branches, merge commits, conflict handling, schema evolution, remotes, ordered scans, crash fault injection or concurrent workloads. The use of atomic SQLite transactions does not, by itself, demonstrate all the durability and operational guarantees needed in production.

Measurement and inference: we cannot infer storage-engine speed from these timings, given the API/CLI boundaries and different resident states. Ten-millisecond sampling can miss a short peak or a child process's counters. The shared oracle data also occupies memory within the worker, so whole-worker RSS is not a measure of product-only memory. Counts of keys, log operations and trie-node pairs refer to different work; Dolt does not expose its internal work here. Our intervals describe repeated runs of fixed histories and have no multiple-comparison correction. Checking the source and hashes establishes internal integrity, not an independent reproduction of performance.

Selection: $T=16,384$ was the largest candidate in the bounded historical search. The cloud controls do not locate a crossover or an optimal threshold. In particular, the slower choice at 32,768 operations remains part of the results. Further experiments should hold row count and history length fixed while varying locality and density independently. They should control process/hash-seed conditions and use independent histories as the unit of uncertainty. A replacement for the frozen policy would need its own calibration followed by held-out validation.

Possible storage changes include binary nodes, value chunks, packed objects, compression and reachability-based garbage collection. Range operations could be supported through an ordered Merkle index or content-defined chunking. Each proposal needs its own semantic and performance evaluation; its effects cannot be inferred from the fixed-trie results presented here.

8. Conclusion. Revon supports two semantically consistent version-diff paths over incremental updates to a fixed trie, addressed changesets and atomic SQLite persistence. On a cloud machine, the repaired source passed 516 Linux workflow executions and a separate 72-execution supplement covering geometry and telemetry. Repeated M/H comparisons show benefits from the log path in selected workloads, with parity still included in the dense case's interval. The frozen threshold sometimes selects a path slower than forced log, leaving optimal adaptation unestablished. One run per system at a million records verifies bounded feasibility and records resource costs, without supporting a comparative scaling claim. This work contributes an auditable prototype and a trade-off study of complete versioning workflows. Its broader generality and independent reproduction remain open.

Funding. No funding was received by the authors for this work.

Competing interests. The authors have no competing interests to declare.

Author contributions. Atharva Sheersh Pandey performed 80% of the research work, and Adhyan Jain performed 20%. Guidance and critical review were provided by Poornima Nedunchezian.

Ethics and consent to participate. This computational study recruited no participants and performed no human or animal intervention. It uses synthetic histories and publicly released de-identified TLC records. We make no claim of institutional ethics approval or

exemption.

Consent for publication. No identifiable participant material is included, so consent for publication is not applicable.

Data and code availability. NYC TLC provides public access to the source file [22]. Its retained checksum and preparation procedure identify the input used here. Source code, archived experiment snapshots and experimental evidence are available at <https://github.com/atharvasheersh/Revon>. The GitHub reproducibility archive and reproduction instructions are described in Appendix A. Raw TLC records and generated databases are not redistributed.

Acknowledgment. We thank the open-source Dolt project and the researchers whose studies of dataset versioning helped shape our experimental design.

REFERENCES

- [1] M. AKHLAGHI ET AL. (2021). Toward long-term and archivable reproducibility, *Computing in Science & Engineering*, vol. 23, no. 3, pp. 82-91, <https://doi.org/10.1109/MCSE.2021.3072860>. [Online]. Available: <https://arxiv.org/abs/2006.03018>.
- [2] M. ARMBRUST ET AL. (2020). Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores, *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3411-3424, <https://doi.org/10.14778/3415478.3415560>.
- [3] ATTIC LABS. Noms Technical Overview. GitHub. [Online]. Available: <https://github.com/attic-labs/noms/blob/master/doc/intro.md>.
- [4] A. AUVOLAT AND F. TAÏANI (2019). Merkle Search Trees: Efficient State-Based CRDTs in Open Networks, in *Proc. 38th IEEE Int. Symp. Reliable Distributed Syst. (SRDS)*, pp. 221-230, <https://doi.org/10.1109/SRDS47363.2019.00032>.
- [5] J. BENET (2014). IPFS: Content Addressed, Versioned, P2P File System, arXiv:1407.3561, [Online]. Available: <https://arxiv.org/abs/1407.3561>.
- [6] A. BHARDWAJ ET AL. (2015). DataHub: Collaborative Data Science and Dataset Version Management at Scale, in *Proc. 7th Biennial Conf. Innovative Data Syst. Research (CIDR)*, Open copy: https://www.cidrdb.org/cidr2015/Papers/CIDR15_Paper18.pdf.
- [7] S. BHATTACHERJEE, A. CHAVAN, S. HUANG, A. DESHPANDE, AND A. PARAMESWARAN (2015). Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff, *Proc. VLDB Endowment*, vol. 8, no. 12, pp. 1346-1357, <https://doi.org/10.14778/2824032.2824035>.
- [8] G. S. BRODAL, S. SIOUTAS, K. TSAKALIDIS, AND K. TSICHLAS (2020). Fully persistent B-trees, *Theoretical Computer Science*, vol. 841, pp. 10-26, <https://doi.org/10.1016/j.tcs.2020.06.027>. [Online]. Available: <https://cs.au.dk/~gerth/papers/tcs20.pdf>.
- [9] P. BUNEMAN, S. KHANNA, AND W.-C. TAN (2001). Why and where: A characterization of data provenance, in *Database Theory: ICDT Lecture Notes in Computer Science*, vol. 1973, 2001, pp. 316-330, https://doi.org/10.1007/3-540-44503-X_20. [Online]. Available: https://www.pure.ed.ac.uk/ws/files/16509989/Why_and_Where_A_Characterization_of_Data_Provenance.pdf.
- [10] S. CHACON AND B. STRAUB (2014). Git Internals: Git Objects, in *Pro Git*, 2nd ed., [Online]. Available: <https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>.
- [11] DOLTHUB (2026). Dolt Storage Engine: Block Store, Dolt Documentation, accessed Oct. 2, [Online]. Available: <https://www.dolthub.com/docs/architecture/storage-engine/block-store/>.
- [12] J. R. DRISCOLL, N. SARNAK, D. D. SLEATOR, AND R. E. TARJAN (1989). Making data structures persistent, *J. Comput. Syst. Sci.*, vol. 38, no. 1, pp. 86-124, [https://doi.org/10.1016/0022-0000\(89\)90034-2](https://doi.org/10.1016/0022-0000(89)90034-2).
- [13] J. EDWARDS AND T. PETRICEK (2025). Baseline: Operation-based evolution and versioning of data, arXiv:2512.09762, preprint, updated 2026. [Online]. Available: <https://arxiv.org/abs/2512.09762>.
- [14] A. GONZÁLEZ-CEBRIÁN, M. BRADFORD, A. E. CHIS, AND H. GONZÁLEZ-VÉLEZ (2024). Standardised versioning of datasets: A FAIR-compliant proposal, *Scientific Data*, vol. 11, art. 358, <https://doi.org/10.1038/s41597-024-03153-y>. [Online]. Available:

- <https://www.nature.com/articles/s41597-024-03153-y>.
- [15] H. GOU, Z. ZHANG, Y. SUN, P. XU, F. TIAN, L. WANG, AND J. WANG (2026). Git4Data: Database-native version control for AI agents, arXiv:2609.02106, preprint. [Online]. Available: <https://arxiv.org/abs/2609.02106>.
 - [16] A. GUO, J. LI, P. SUKPRASERT, S. KHULLER, A. DESHPANDE, AND K. MUKHERJEE (2024). To store or not to store: A graph theoretical approach for dataset versioning, in Proc. IEEE Int. Parallel and Distributed Processing Symposium (IPDPS), pp. 479-493, <https://doi.org/10.1109/IPDPS57955.2024.00049>. [Online]. Available: <https://arxiv.org/abs/2402.11741>.
 - [17] S. HUANG, L. XU, J. LIU, A. J. ELMORE, AND A. PARAMESWARAN (2017). OrpheusDB: Bolt-on Versioning for Relational Databases, Proc. VLDB Endowment, vol. 10, no. 10, pp. 1130-1141, <https://doi.org/10.14778/3115404.3115417>.
 - [18] M. MADDOX ET AL. (2016). Decibel: The Relational Dataset Branching System, Proc. VLDB Endowment, vol. 9, no. 9, pp. 624-635, <https://doi.org/10.14778/2947618.2947619>.
 - [19] F. MCSHERRY, D. G. MURRAY, R. ISAACS, AND M. ISARD (2013). Differential dataflow, in Proc. 6th Biennial Conf. Innovative Data Systems Research (CIDR), [Online]. Available: https://www.cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf.
 - [20] R. C. MERKLE (1988). A digital signature based on a conventional encryption function, in Advances in Cryptology: CRYPTO '87, Lecture Notes in Computer Science, vol. 293, pp. 369-378, https://doi.org/10.1007/3-540-48184-2_32.
 - [21] A. MUTHITACHAROEN, B. CHEN, AND D. MAZIÈRES (2001). A low-bandwidth network file system, in Proc. 18th ACM Symp. Operating Syst. Principles (SOSP), pp. 174-187, <https://doi.org/10.1145/502034.502052>.
 - [22] NEW YORK CITY TAXI AND LIMOUSINE COMMISSION (2024). TLC Trip Record Data: Yellow Taxi, January public dataset, accessed Oct. 1, 2026. [Online]. Available: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>.
 - [23] N. SAQUIB, C. KRINTZ, AND R. WOLSKI (2023). Replicated versioned data structures for wide-area distributed systems, IEEE Transactions on Parallel and Distributed Systems, vol. 34, no. 1, pp. 207-224, <https://doi.org/10.1109/TPDS.2022.3217969>. [Online]. Available: <https://racelab.cs.ucsb.edu/papers/tpds22-versioned.pdf>.
 - [24] M. E. SCHÜLE, L. KARNOWSKI, J. SCHMEIßER, B. KLEINER, A. KEMPER, AND T. NEUMANN (2019). Versioning in main-memory database systems: From MusaeusDB to TardisDB, in Proc. 31st Int. Conf. Scientific and Statistical Database Management (SSDBM), pp. 169-180, <https://doi.org/10.1145/3335783.3335792>. [Online]. Available: <https://www-db.cs.tum.edu/~schuele/data/tardis.pdf>.
 - [25] A. SEERING, P. CUDRÉ-MAUROUX, S. MADDEN, AND M. STONEBRAKER (2012). Efficient versioning for scientific array databases, in Proc. IEEE 28th Int. Conf. Data Engineering (ICDE), pp. 1013-1024, <https://doi.org/10.1109/ICDE.2012.102>. [Online]. Available: <https://dspace.mit.edu/entities/publication/a5c2c050-6abd-47c1-8bee-4f8c51b6a15f>.
 - [26] T. SEHN (2024). Dolt's Storage Engine, DoltHub Blog, Feb. 29, [Online]. Available: <https://www.dolthub.com/blog/2024-02-29-storage-engine/>.
 - [27] R. SHRAGA AND R. J. MILLER (2023). Explaining Dataset Changes for Semantic Data Versioning with Explain-Da-V, Proc. VLDB Endow., vol. 16, no. 6, pp. 1587-1600, <https://doi.org/10.14778/3583140.3583169>.
 - [28] E. SOROUSH AND M. BALAZINSKA (2013). Time travel in a scientific array database, in Proc. IEEE 29th Int. Conf. Data Engineering (ICDE), pp. 98-109, <https://doi.org/10.1109/ICDE.2013.6544817>. [Online]. Available: https://scidb.cs.washington.edu/paper/ICDE13_conf_full_422.pdf.
 - [29] A. TWIGG, A. BYDE, G. MIŁOS, T. MORETON, J. WILKES, AND T. WILKIE (2011). Stratified B-trees and versioning dictionaries, in Proc. 3rd USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage), [Online]. Available: <https://arxiv.org/abs/1103.4282>.
 - [30] S. WANG ET AL. (2018). ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications, Proc. VLDB Endowment, vol. 11, no. 10, pp. 1137-1150, <https://doi.org/10.14778/3231751.3231762>.
 - [31] J. ZHAO AND S. KRISHNAN (2024). Compression and in-situ query processing for fine-grained array lineage, arXiv:2405.17701, [Online]. Available: <https://arxiv.org/abs/2405.17701>.

APPENDIX A: REPRODUCIBILITY

Readers can find the code and experiment records in the [Revon GitHub repository](#). The `paper/` directory contains [Revon_Linux_Reproducibility_Supplement.zip](#). This archive brings together the source versions used in the runs, fixed protocols, manifests and raw measurements. It also includes the paired intervals, semantic checks and audit records. The README files explain how to set up the dependencies, prepare the data, run the experiments and audit the evidence [1].

The `linux-validation-20261008` and `linux-final-supplement-20261008` bundles cover different experiments, so their timing measurements should be analysed separately. We do not redistribute the TLC records or generated repositories; readers must rebuild them using the documented steps. To audit the archived results, the records and metadata must agree with the saved `dataset.json`. Run the auditors on a disposable copy of the evidence, since they replace `audit.json`. These checks establish whether the recorded results are internally consistent. They do not independently reproduce the timing experiment, and identical semantic hashes do not guarantee the same elapsed times.